

Inception correction

Attachments

 [subject.pdf](#)

Preliminaries

If cheating is suspected, the evaluation stops here. Use the "Cheat" flag to report it. Take this decision calmly, wisely, and please, use this button with caution.

Preliminary tests

- Any credentials, API keys, environment variables must be set inside a .env file during the evaluation. In case any credentials, API keys are available in the git repository and outside of the .env file created during the evaluation, the evaluation stop and the mark is 0.
- Defense can only happen if the evaluated student or group is present. This way everybody learns by sharing knowledge with each other.
- If no work has been submitted (or wrong files, wrong directory, or wrong filenames), the grade is 0, and the evaluation process ends.
- For this project, you have to clone their Git repository on their station.

 Yes

 No

General instructions

General instructions

- For the entire evaluation process, if you don't know how to check a requirement, or verify anything, the evaluated student has to help you.
- Ensure that all the files required to configure the application are located inside a srcs folder. The srcs folder must be located at the root of the repository.
- Ensure that a Makefile is located at the root of the repository.
- Before starting the evaluation, run this command in the terminal: "docker stop \$(docker ps -qa); docker rm \$(docker ps -qa); docker rmi -f \$(docker images -qa); docker volume rm \$(docker volume ls -q); docker network rm \$(docker network ls -q) 2>/dev/null"
- Read the docker-compose.yml file. There mustn't be 'network: host' in it or 'links:'. Otherwise, the evaluation ends now.
- Read the docker-compose.yml file. There must be 'network(s)' in it. Otherwise, the evaluation ends now.
- Examine the Makefile and all the scripts in which Docker is used. There mustn't be '--link' in any of them. Otherwise, the evaluation ends now.
- Examine the Dockerfiles. If you see 'tail -f' or any command run in background in any of them in the ENTRYPOINT section, the evaluation ends now. Same thing if 'bash' or 'sh' are used but not for running a script (e.g, 'nginx & bash' or 'bash').
- If the entrypoint is a script (e.g., ENTRYPOINT ["sh", "my_entrypoint.sh"], ENTRYPOINT ["bash", "my_entrypoint.sh"]), ensure it runs no program in background (e.g, 'nginx & bash').
- Examine all the scripts in the repository. Ensure none of them runs an infinite loop. The following are a few examples of prohibited commands: 'sleep infinity', 'tail -f /dev/null', 'tail -f /dev/random'
- Run the Makefile.

✓ Yes

✗ No

The command that should be executed in the 4th point : "docker stop \$(docker ps -qa); docker rm \$(docker ps -qa); docker rmi -f \$(docker images -qa); docker volume rm \$(docker volume ls -q); docker network rm \$(docker network ls -q) 2>/dev/null"

Mandatory part

This project consists in setting up a small infrastructure composed of different services using docker compose. Ensure that all the following points are correct.

Project overview

- The evaluated person has to explain to you in simple terms:
 - How Docker and docker compose work
 - The difference between a Docker image used with docker compose and without docker compose
 - The benefit of Docker compared to VMs
 - The pertinence of the directory structure required for this project (an example is provided in the subject's PDF file).

✓ Yes

✗ No

Simple setup

- Ensure that NGINX can be accessed by port 443 only. Once done, open the page.
- Ensure that a SSL/TLS certificate is used.
- Ensure that the WordPress website is properly installed and configured (you shouldn't see the WordPress Installation page). To access it, open <https://login.42.fr> in your browser, where login is the login of the evaluated student. You shouldn't be able to access the site via <http://login.42.fr>. If something doesn't work as expected, the evaluation process ends now.

✓ Yes

✗ No

1. How Docker and Docker Compose Work

Docker is a platform that allows you to create, deploy, and run applications in containers. Containers are lightweight, standalone packages that include everything needed to run a piece of software, including the code, runtime, libraries, and system tools. Docker containers are isolated from each other and from the host system, ensuring that they run consistently across different environments.

Docker Compose is a tool that allows you to define and manage multi-container Docker applications. With Docker Compose, you can use a YAML file to configure your application's services, networks, and volumes, and then start all the services with a single command.

Example:

- **Docker:** Imagine you have a Python application that needs a specific version of Python and some libraries. You can create a Docker image that includes Python and the required libraries. When you run this image, it creates a container where your application runs in an isolated environment.
- **Docker Compose:** Now, suppose your Python application needs a database like PostgreSQL and a caching service like Redis. Instead of running each service

manually, you can define all these services in a `docker-compose.yml` file. When you run `docker-compose up`, Docker Compose will start the Python application, PostgreSQL, and Redis containers all at once, and link them together.

2. The Difference Between a Docker Image Used with Docker Compose and Without Docker Compose

Docker Image: A Docker image is a standalone package that includes everything needed to run a piece of software. You can run a Docker image directly using the `docker run` command, which creates a container from the image.

Docker Compose: When using Docker Compose, you still use Docker images, but you define how multiple containers (based on different images) should work together in a `docker-compose.yml` file. Docker Compose manages the lifecycle of these containers, including starting, stopping, and linking them.

Example:

- **Without Docker Compose:** You have a web application that uses a MySQL database. You would need to manually start the MySQL container and then start the web application container, ensuring they are linked correctly.

```
bash
Copy
docker run -d --name mysql-container mysql
docker run -d --name web-app-container --link mysql-container:mysql
web-app-image
```

- **With Docker Compose:** You define both the web application and MySQL service in a `docker-compose.yml` file. Docker Compose handles starting both containers and linking them.

```
yaml
Copy
version: '3'
services:
  web:
    image: web-app-image
    depends_on:
      - db
  db:
    image: mysql
```

Then, you simply run:

```
bash  
Copy  
docker-compose up
```

3. The Benefit of Docker Compared to VMs

Docker and **Virtual Machines (VMs)** both provide isolated environments for running applications, but they do so in different ways.

Virtual Machines (VMs): A VM includes a full copy of an operating system, the application, necessary binaries, and libraries. This makes VMs large in size and resource-intensive. Each VM runs on a hypervisor, which allows multiple VMs to run on a single physical machine.

Docker Containers: Docker containers share the host system's kernel and isolate the application processes from the rest of the system. Containers are much lighter than VMs because they don't need a full OS for each instance. They only include the application and its dependencies.

Benefits of Docker over VMs:

- **Resource Efficiency:** Containers are lightweight and start almost instantly because they share the host OS kernel. VMs, on the other hand, require more resources and take longer to start because they include a full OS.
- **Portability:** Docker containers can run on any system that supports Docker, ensuring consistency across development, testing, and production environments. VMs are less portable due to their size and the need for a compatible hypervisor.
- **Scalability:** Docker makes it easier to scale applications horizontally by spinning up multiple containers. VMs are more cumbersome to scale due to their size and resource requirements.

Example:

- **VM:** Imagine you have a server running three VMs, each with its own OS, running a web server, a database, and a caching service. Each VM consumes a significant amount of disk space, memory, and CPU.
- **Docker:** On the same server, you could run three Docker containers for the web server, database, and caching service. These containers share the host OS kernel, so they use far fewer resources than VMs. You could easily run more containers on the same server, making better use of your hardware.

Docker Basics

- Start by checking the Dockerfiles. There must be one Dockerfile per service. Ensure that the Dockerfiles are not empty files. If it's not the case or if a Dockerfile is missing, the evaluation process ends now.
- Make sure the evaluated student has written their own Dockerfiles and built their own Docker images. Indeed, it is forbidden to use ready-made ones or to use services such as DockerHub.
- Ensure that every container is built from the penultimate stable version of Alpine/Debian. If a Dockerfile does not start with 'FROM alpine:X.X.X' or 'FROM debian:XXXXX', or any other local image, the evaluation process ends now.
- The Docker images must have the same name as their corresponding service. Otherwise, the evaluation process ends now.
- Ensure that the Makefile has set up all the services via docker compose. This means that the containers must have been built using docker compose and that no crash happened. Otherwise, the evaluation process ends.

✓ Yes

✗ No

Docker Network

- Ensure that docker-network is used by checking the docker-compose.yml file. Then run the 'docker network ls' command to verify that a network is visible.
- The evaluated student has to give you a simple explanation of docker-network. If any of the above points is not correct, the evaluation process ends now.

✓ Yes

✗ No

Docker Network is a feature that allows Docker containers to communicate with each other and with the outside world. When you create a Docker network, it acts like a virtual network where containers can be connected.

NGINX with SSL/TLS

- Ensure that there is a Dockerfile.
- Using the 'docker compose ps' command, ensure that the container was created (using the flag '-p' is authorized if necessary).
- Try to access the service via http (port 80) and verify that you cannot connect.
- Open <https://login.42.fr/> in your browser, where login is the login of the evaluated student. The displayed page must be the configured WordPress website (you shouldn't see the WordPress Installation page).
- The use of a TLS v1.2/v1.3 certificate is mandatory and must be demonstrated. The SSL/TLS certificate doesn't have to be recognized. A self-signed certificate warning may appear. If any of the above points is not clearly explained and correct, the evaluation process ends now.

✓ Yes

✗ No

WordPress with php-fpm and its volume

- Ensure that there is a Dockerfile.
- Ensure that there is no NGINX in the Dockerfile.
- Using the 'docker compose ps' command, ensure that the container was created (using the flag '-p' is authorized if necessary).
- Ensure that there is a Volume. To do so: Run the command 'docker volume ls' then 'docker volume inspect <volume name>'. Verify that the result in the standard output contains the path '/home/login/data/', where login is the login of the evaluated student.
- Ensure that you can add a comment using the available WordPress user.
- Sign in with the administrator account to access the Administration dashboard. The Admin username must not include 'admin' or 'Admin' (e.g., admin, administrator, Admin-login, admin-123, and so forth).
- From the Administration dashboard, edit a page. Verify on the website that the page has been updated. If any of the above points is not correct, the evaluation process ends now.

✓ Yes

✗ No

To access the login form: <https://login.42.fr/wp-config.php>

MariaDB and its volume

- Ensure that there is a Dockerfile.
- Ensure that there is no NGINX in the Dockerfile.
- Using the 'docker compose ps' command, ensure that the container was created (using the flag '-p' is authorized if necessary).
- Ensure that there is a Volume. To do so: Run the command 'docker volume ls' then 'docker volume inspect <volume name>'. Verify that the result in the standard output contains the path '/home/login/data/', where login is the login of the evaluated student.
- The evaluated student must be able to explain you how to login into the database. Verify that the database is not empty. If any of the above points is not correct, the evaluation process ends now.

✓ Yes

✗ No

Persistence!

- This part is pretty straightforward. You have to reboot the virtual machine. Once it has restarted, launch docker compose again. Then, verify that everything is functional, and that both WordPress and MariaDB are configured. The changes you made previously to the WordPress website should still be here. If any of the above points is not correct, the evaluation process ends now.

✓ Yes

✗ No

How to login to the mariadb database?

1- write in the terminal : “mariadb -h 127.0.0.1 -P 3306 -u root -p12345”

-h 127.0.0.1: the host

-P 3306: the port of mariadb

-u root: the username (here is root)

-p12345: the root password (check it before)

2- SHOW DATABASES;

3- USE my_db;

4- SHOW TABLES;

Rebooting the machine: sudo reboot

Here is the .env file that I created:

```
# MYSQL SETUP

MYSQL_DB=my_db
MYSQL_USER=jfatfat
MYSQL_PASSWORD=jfatfat123
MYSQL_ROOT_PASSWORD=12345

# WORDPRESS SETUP

DOMAIN_NAME=jfatfat.42.fr
WP_ADMIN_NAME=jfatfat
WP_ADMIN_PASS=jihaD
WP_TITLE=jfatfat_website
WP_ADMIN_EMAIL=jihaDmohfatfat1999@gmail.com

# WORDPRESS NEW USER SETUP

WP_USER_NAME=user1
WP_USER_PASS=password1
WP_USER_ROLE=author
WP_USER_EMAIL=jihaDfatfatt99@gmail.com
```